

UART Receiver Using Finite State Machine (FSM)

Title: Design and Implementation of a UART Receiver Using Finite State Machine (FSM) in Verilog HDL

ABSTRACT

Universal Asynchronous Receiver Transmitter (UART) is a widely used serial communication protocol for exchanging data between digital devices. This project presents the design and implementation of a UART Receiver using a Finite State Machine (FSM) in Verilog HDL. The receiver detects the start bit, receives 8-bit serial data, verifies the stop bit, and generates the received parallel data along with a data_valid signal. A framing_error signal is also generated if the stop bit is invalid.

The design was functionally verified using Icarus Verilog, and the simulation waveform was analyzed using GTKWave. The hardware synthesis was performed using Yosys, and the FPGA resource utilization was obtained. The synthesized design uses 33 LUT4s, 24 Flip-Flops, and 3 Carry Cells, indicating an efficient hardware implementation. The project demonstrates how an FSM can be used to implement a reliable UART receiver and serves as a foundation for digital communication system design.

1. INTRODUCTION

Serial communication is one of the most common methods used for transferring data between electronic devices. Unlike parallel communication, serial communication sends one bit of data at a time through a single communication line, reducing hardware complexity and wiring cost.

UART (Universal Asynchronous Receiver Transmitter) is a simple serial communication protocol that does not require a common clock signal between the transmitting and receiving devices. Instead, the transmitter and receiver follow the same communication timing.

A UART communication system consists of two parts:

UART Transmitter

UART Receiver

In this project, only the UART Receiver is implemented. The receiver continuously monitors the serial input line (RX). When a start bit is detected, it receives eight serial bits one by one, stores them in a shift register, verifies the stop bit, and transfers the received byte to the output. The complete operation is controlled using a Finite State Machine (FSM).

The design was written in Verilog HDL, simulated using Icarus Verilog, verified using GTKWave, and synthesized using Yosys.

2. PROBLEM STATEMENT

In digital communication systems, data is often transmitted serially to reduce the number of communication lines. A receiver must correctly identify the beginning of a data frame, receive each incoming bit in the correct order, verify that the transmission is complete, and then provide the received data to the system.

Designing such a receiver requires proper control logic to ensure reliable data reception.

Therefore, the objective of this project is to design and verify a UART Receiver using a Finite State Machine (FSM) capable of receiving serial data and converting it into 8-bit parallel data.

3. OBJECTIVES

- The objectives of this project are:
- To understand the UART communication protocol.
- To design a UART Receiver using Verilog HDL.
- To implement the receiver using a Finite State Machine (FSM).
- To receive serial data correctly.
- To verify the design using Icarus Verilog.
- To observe the output waveform using GTKWave.
- To synthesize the design using Yosys and analyze hardware resource utilization

4. SOFTWARE TOOLS USED

- Verilog HDL : Hardware description language used to design the UART Receiver.
- Icarus Verilog : Compiles and simulates the Verilog design.
- GTKWave : Displays and analyzes the simulation waveform.
- Yosys : Synthesizes the Verilog design and generates resource utilization.
- Ubuntu (WSL :E nvironment used to run the design tools.

5. METHODOLOGY

The UART Receiver was implemented using a Finite State Machine (FSM). The receiver continuously monitors the RX input line. When the start bit is detected, the FSM changes its state and begins receiving serial data.

The received bits are stored one by one in an 8-bit shift register. After all eight bits are received, the receiver checks whether the stop bit is valid. If the stop bit is correct, the received byte is transferred to the output, and the data_valid signal is asserted. If the stop bit is invalid, the framing_error signal is generated.

The completed design is compiled using Icarus Verilog, verified using GTKWave, and synthesized using Yosys to analyze hardware resource utilization.

6. UART RECEIVER FRAME FORMAT

The UART receiver processes the incoming serial data in the following format:

Start Bit	Data Bits	Stop Bit
1 Bit (0)	8 Bits	1 Bit (1)

Start Bit: Indicates the beginning of data transmission. It is always LOW (0).

Data Bits: Eight bits of serial data are transmitted one after another.

Stop Bit: Indicates the end of transmission. It is always HIGH (1).

In this project, the receiver detects the start bit, receives the 8-bit data, verifies the stop bit, and then outputs the received byte.

7. FINITE STATE MACHINE (FSM)

A Finite State Machine (FSM) is a digital design technique used to control sequential operations. Instead of performing all operations at once, the system moves through a sequence of predefined states.

The UART receiver in this project uses an FSM to control the reception of serial data.

FSM States

- **IDLE** : Waits for the start bit.
- **START** : Confirms the start bit and prepares to receive data.
- **DATA** : Receives 8 serial data bits and stores them in the shift register.
- **STOP** : Checks whether the stop bit is HIGH.
- **DONE** : Transfers the received data to the output and asserts the data_valid signal.

8. WORKING OF THE UART RECEIVER

The UART receiver continuously monitors the RX input line.

Step 1: Idle State

Initially, the receiver remains in the IDLE state. During this state, the RX line remains HIGH.

Step 2: Start Bit Detection

When the RX line changes from HIGH to LOW, the receiver detects the start bit and moves to the START state.

Step 3: Data Reception

After detecting the start bit, the receiver enters the DATA state. It receives one serial bit during each clock cycle and stores it in the 8-bit shift register.

A counter keeps track of the number of bits received. After receiving eight bits, the receiver moves to the next state.

Step 4: Stop Bit Verification

The receiver checks whether the stop bit is HIGH.

If the stop bit is HIGH, the received data is considered valid.

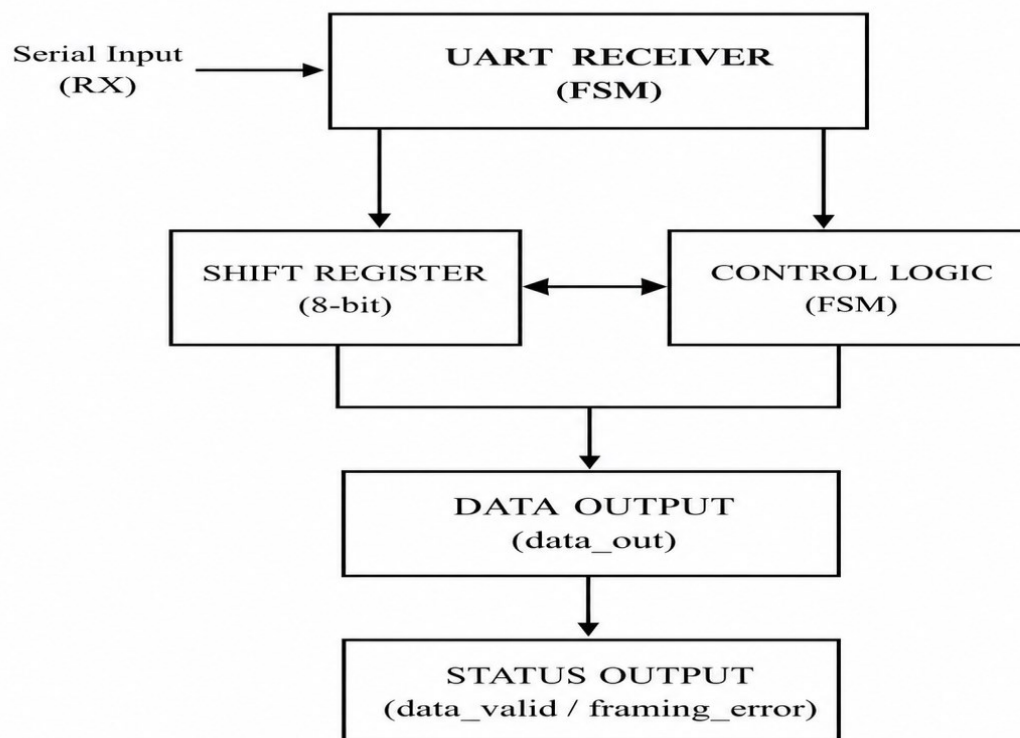
If the stop bit is LOW, the framing_error signal becomes HIGH.

Step 5: Data Output

The received byte is transferred to data_out, and the data_valid signal becomes HIGH, indicating successful reception.

Finally, the receiver returns to the IDLE state to receive the next byte

9.Block Diagram



10.Verilog Implementation

UART Receiver(uart_rx.v)

```
module uart_rx(  
    input wire clk,
```

```

input wire rst,
input wire rx,
output reg [7:0] data_out,
output reg data_valid,
output reg framing_error
);
// FSM States
parameter IDLE = 3'b000;
parameter START = 3'b001;
parameter DATA = 3'b010;
parameter STOP = 3'b011;
parameter DONE = 3'b100;

reg [2:0] state;
reg [2:0] bit_count;
reg [7:0] shift_reg;

always @(posedge clk or posedge rst)
begin
    if (rst)
    begin
        state <= IDLE;
        bit_count <= 3'd0;
        shift_reg <= 8'd0;
        data_out <= 8'd0;
        data_valid <= 1'b0;
        framing_error <= 1'b0;
    end
    else
    begin

```

```
case(state)
```

```
    IDLE:
```

```
    begin
```

```
        data_valid <= 1'b0;
```

```
        framing_error <= 1'b0;
```

```
        if(rx == 1'b0)
```

```
            state <= START;
```

```
    end
```

```
    START:
```

```
    begin
```

```
        bit_count <= 3'd0;
```

```
        state <= DATA;
```

```
    end
```

```
    DATA:
```

```
    begin
```

```
        shift_reg[bit_count] <= rx;
```

```
        if(bit_count == 3'd7)
```

```
            state <= STOP;
```

```
        else
```

```
            bit_count <= bit_count + 1;
```

```
    end
```

```
    STOP:
```

```
    begin
```

```
        if(rx == 1'b1)
```

```

        begin
            data_out <= shift_reg;
            data_valid <= 1'b1;
            framing_error <= 1'b0;
        end
    else
        begin
            framing_error <= 1'b1;
        end

        state <= DONE;
    end

    DONE:
        begin
            state <= IDLE;
        end

    default:
        state <= IDLE;

    endcase
end

end

endmodule

Testbench(tb_uart_rx.v)
`timescale 1ns/1ps

module tb_uart_rx;

```

```

reg clk;

reg rst;

reg rx;


wire [7:0] data_out;
wire data_valid;
wire framing_error;


uart_rx uut (
    .clk(clk),
    .rst(rst),
    .rx(rx),
    .data_out(data_out),
    .data_valid(data_valid),
    .framing_error(framing_error)
);


// Clock generation (10 ns period)
always #5 clk = ~clk;


// Task to send one UART byte
task send_byte;
    input [7:0] data;
    integer i;
    begin
        // Start bit
        rx = 0;
        #10;

        // Data bits (LSB first)
        for(i = 0; i < 8; i = i + 1)
            begin

```



```

        rx = data[i];

        #10;
    end

    // Stop bit
    rx = 1;

    #10;
end
endtask

initial begin

    $dumpfile("uart.vcd");
    $dumpvars(0, tb_uart_rx);

    clk = 0;
    rst = 1;
    rx = 1;

    #20;
    rst = 0;

    // Send 0xA5 (10100101)
    send_byte(8'hA5);

    #100;

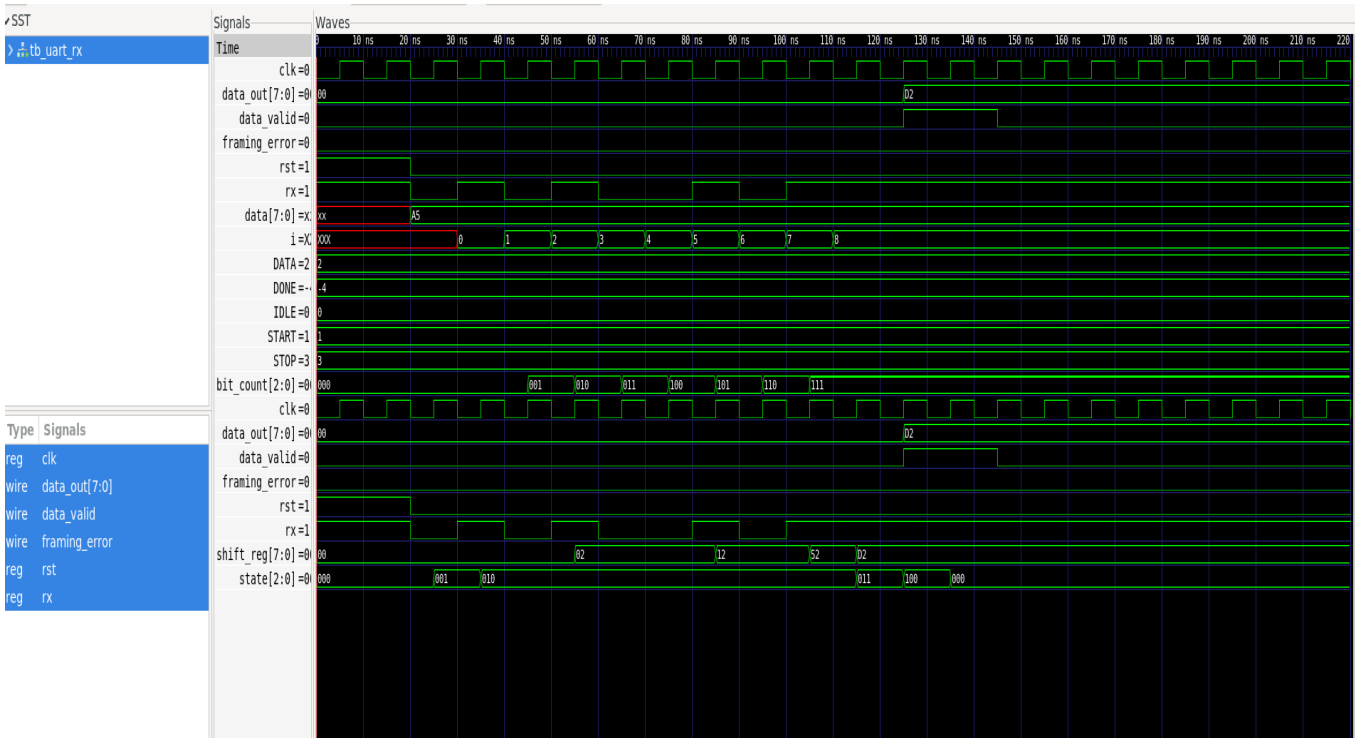
    $finish;

end

endmodule

```

12.Simulation Results



The simulation waveform verifies the correct operation of the UART Receiver. Initially, the receiver remains in the IDLE state. When the RX line goes LOW, the start bit is detected, and the FSM begins receiving the serial data bits. The received bits are stored in the shift register. After the stop bit is verified, the received byte is transferred to data_out, and the data_valid signal becomes HIGH. Since the stop bit is valid, the framing_error signal remains LOW.

10. SYNTHESIS RESULTS

Yosys statistics

```

5.9. Finished OPT passes. (There is nothing left to do.)

yosys> stat

6. Printing statistics.

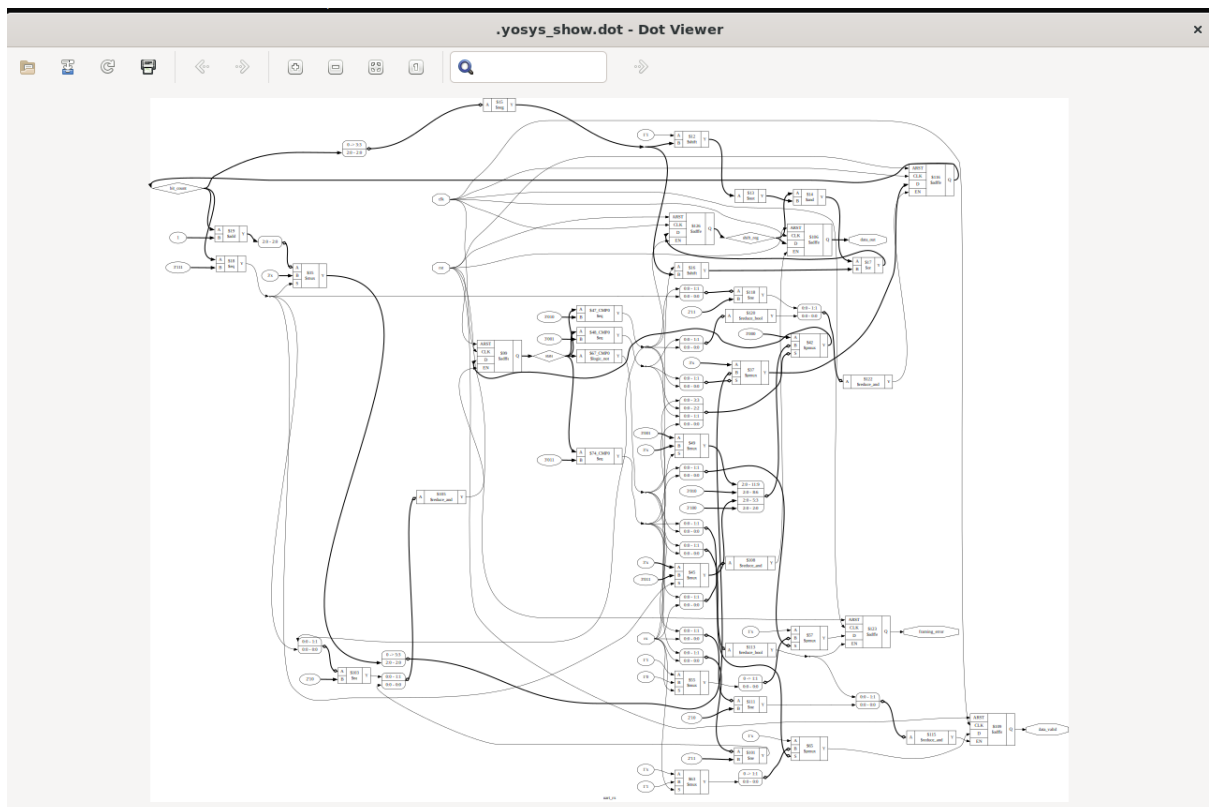
=== uart_rx ===

Number of wires:          40
Number of wire bits:      137
Number of public wires:   9
Number of public wire bits: 27
Number of ports:          6
Number of port bits:      13
Number of memories:       0
Number of memory bits:    0
Number of processes:      0
Number of cells:          37
  $add                    1
  $adffe                  6
  $and                    1
  $eq                     4
  $logic_not              1
  $mux                    5
  $ne                     4
  $neg                    1
  $not                    1
  $or                     1
  $pmux                   4
  $reduce_and             4
  $reduce_bool            2
  $shift                  2

yosys> show
7. Generating Graphviz representation of design.
Writing dot description to `~/home/kushi/.yosys_show.dot'.
Dumping module uart_rx to page 1.
Exec: { test -f '/home/kushi/.yosys_show.dot.pid' && fuser -s '/home/kushi/.yosys_show

```

RTL Diagram



FPGA Resource Utilization

2.48. Printing statistics.

```
=== uart_rx ===
```

Number of wires:	33
Number of wire bits:	93
Number of public wires:	33
Number of public wire bits:	93
Number of ports:	6
Number of port bits:	13
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	60
SB_CARRY	3
SB_DFFER	24
SB_LUT4	33

2.49. Executing CHECK pass (checking for obvious problems).

```
Checking module uart_rx...
```

```
Found and reported 0 problems.
```

11. APPLICATIONS

- Serial communication systems
- Embedded systems
- FPGA-based digital designs
- Microcontroller communication
- Industrial control systems
- IoT devices

12. ADVANTAGES

- Simple and modular design
- Efficient FSM-based implementation
- Reliable serial data reception
- Easy to simulate and verify
- Suitable for FPGA synthesis

13. FUTURE ENHANCEMENTS

- Add configurable baud-rate support.
- Implement parity bit checking.

- Design a UART transmitter module.
- Support variable data lengths.
- Implement complete UART communication.

14. CONCLUSION

A UART Receiver using a Finite State Machine (FSM) was successfully designed and implemented using Verilog HDL. The design correctly receives serial data, verifies the stop bit, and generates the corresponding parallel output. Functional verification was carried out using Icarus Verilog and GTKWave, while Yosys synthesis confirmed efficient hardware utilization. The project demonstrates the practical implementation of UART communication using digital design principles and provides a strong foundation for future enhancements.